Software Test Documentation (STD)

Project Information

Project Name: Federated Social Networking Platform with Decentralized Identity
Document Type: Software Testing Document (IEEE 829 Standard)
Date: March 11, 2026
Version: 1.0

---

1. Introduction

1.1 Purpose of the Testing Document

The primary purpose of this Software Testing Document is to outline the testing methodologies, strategies, execution phases, and reporting protocols for the "Federated Social Networking Platform with Decentralized Identity." This document provides a comprehensive framework to ensure the platform meets all functional, non-functional, security, and performance criteria prior to production deployment.

1.2 Scope of Testing

The scope of testing encompasses the validation of all core modules developed for the web application. This includes frontend user interfaces, backend API services, database interactions, inter-node federation mechanisms, and AI-driven content moderation. The testing covers internal business logic, component integration, and complete end-to-end user journeys defined by the project specifications.

1.3 Definitions and Abbreviations

- **API:** Application Programming Interface
- **E2E:** End-to-End
- **UI/UX:** User Interface / User Experience
- **ActivityPub:** A decentralized social networking protocol based on the W3C standard.
- **CI/CD:** Continuous Integration / Continuous Deployment
- **STD:** Software Testing Document
- **TC:** Test Case
- **AES:** Advanced Encryption Standard

---

2. Test Items

The software items and modules subject to testing in this cycle include:

1. **Identity and Authentication Subsystem:** Registration, login, session management, and cryptographic security (AES-256 encryption validation).
2. **Content Sharing Subsystem:** Post creation, media attachment, and interaction mechanisms (likes, replies, stories).
3. **AI Moderation Subsystem:** Real-time payload evaluation utilizing external AI interfaces (Groq API) to detect hate speech and profanity in multiple languages.
4. **Federation Subsystem:** Node-to-node ActivityPub communications, including user resolution, follow requests, and push-based delivery across decentralized servers.
5. **Messaging Subsystem:** Real-time peer-to-peer messaging operations and media payload delivery.
6. **Admin Dashboard:** Access controls, platform traffic monitoring, moderation action logs, and generalized system oversight configurations.

---

3. Features to be Tested

Detailed testing will be conducted on the following specific application features to assure compliance with business requirements:

- **Login & Registration:** Validation of email formats, real-time password strength checks, holographic 3D UI rendering logic, and account initialization.
- **Content Posting:** Creation, persistent storage, retrieval, and deletion of multimedia user posts and ephemeral system "stories".
- **Multilingual Profanity Detection:** Algorithmic detection of harmful language, slurs, and glorification of notorious figures utilizing the dual-channel priority queue AI moderation system.
- **Moderation Rules Enforcement:** Application of blocking popups for offensive profiles and automated post flagging/hiding operations.
- **Federation Communication:** ActivityPub protocol adherence, including correct formatting of WebFinger endpoints and cross-origin Mastodon instance communication.
- **Admin Controls:** Secure dashboard access, execution of account bans, and inspection of AI moderation queue pipelines.

---

4. Features Not to be Tested

The following functionalities and components are explicitly excluded from this testing framework:

- **External Infrastructure Dependencies:** Physical server hardware reliability, operating system-level virtualization stability, and cloud provider (AWS/Vercel/Render) native CDN performance.
- **Third-Party Services Outside System Scope:** The internal operational logic and uptime of external services such as the Groq AI API or Brevo email delivery infrastructure. Testing will only interact with their external-facing APIs to ensure our system handles their responses (and potential timeouts) correctly.

---

5. Test Strategy / Approach

To formulate a reliable and robust system, a multi-tiered testing strategy has been deployed:

5.1 Unit Testing

Focused on isolating the smallest pieces of testable software (e.g., individual Go functions or isolated React components). This phase ensures that core algorithmic logic—such as handle parsing, AES encryption tools, and input validation—functions perfectly.

5.2 Integration Testing

Targeted at validating the interfaces between integrated components. Crucial areas include the communication loop between the application backend and the MongoDB database, or the data handoff between the Moderation Service and the Content Sharing Service.

5.3 System Testing

Evaluation of the completely integrated application to verify that it meets the overall requirements specifications. This involves staging the entire system and verifying architectural components work harmoniously in a production-like setting.

5.4 End-to-End (E2E) Testing

Simulating real-world user workflows from start to finish. E2E testing evaluates the complete user journey, ensuring that network calls, database writes, and frontend state updates occur sequentially and correctly without failure.

5.5 Automated Testing using Playwright

Automated scripts utilizing Playwright are engineered to run daily regression tests. These scripts programmatically drive a headless browser to interact with the DOM, simulating clicks, keyboard input, and form submissions to validate UI persistence across repeated deployments.

---

6. Test Environment

The execution of these tests utilizes a standardized environment designed to parallel the intended production state.

6.1 Hardware Requirements

- **Processor:** Minimum 4-Core CPU (Simulated Cloud Environment)
- **Memory:** 8 GB RAM
- **Storage:** 20 GB SSD

6.2 Software Requirements

- **Operating System:** Windows Server / Linux Ubuntu (22.04 LTS) for CI pipelines.
- **Database:** MongoDB 7.x (Local and Atlas clusters)
- **Runtime Environments:** Node.js (v20+), Go (v1.22+)

6.3 Tools Used

- **Automation Automation & E2E:** Playwright
- **Backend Unit & Integration:** Go native `testing` package
- **Frontend Unit:** Jest & React Testing Library
- **CI/CD Platform:** GitHub Actions

---

7. Test Case Design

The table below details robust test specifications mapped to critical use cases:

Test Case ID	Module	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-001	Authentication	Verify user registration with valid credentials.	1. Navigate to Sign Up. 2. Enter unique email, display name, user handle, and secure password. 3. Submit the form.	System creates account, encrypts email via AES, and redirects to Dashboard.	Account created, AES encryption verified in DB, redirect successful.	**Pass**
TC-002	Authentication	Prevent sign up with duplicate email.	1. Navigate to Sign Up. 2. Enter an email currently existing in the database. 3. Submit the form.	UI displays "Email already in use" validation error. Account is not duplicated.	Warning displayed. DB does not duplicate entry.	**Pass**
TC-003	Moderation	Detect and block offensive profile fields.	1. Navigate to Edit Profile. 2. Input hate speech into the "Bio" field. 3. Click Save.	The AI Module flags the bio, system aborts save, and returns a blocking popup to the user.	AI block triggered successfully. Bio change reverted.	**Pass**
TC-004	Moderation	Process post through dual-channel priority queue.	1. Submit a post with benign text. 2. Immediately submit a post containing profanity.	Post 1 passes silently into DB. Post 2 is intercepted by the high-priority queue and flagged.	High priority queue accurately routed and flagged Post 2.	**Pass**
TC-005	Federation	Resolve external ActivityPub actor.	1. Type external handle (e.g., `@user@mastodon.social`) in search bar. 2. Click resolve.	Backend triggers WebFinger lookup, resolves actor metadata, and serves remote profile UI.	Profiler rendered with remote data.	**Pass**
TC-006	Federation	Send cross-instance Follow	1. Navigate to a resolved remote profile. 2. Click "Follow".	System signs an ActivityPub `Follow` payload and dispatches to	Dispatch successful; 202 Accepted returned	**Pass**

		Request.		remote outbox.	from remote.	
TC-007	Posting	Create text post with media attachment.	1. Enter text in post composer. 2. Attach a `.png` file under 5MB. 3. Click "Post".	Post is saved in DB, media URL generated, CORS rules applied, and timeline dynamically updates.	Media successfully uploaded and displayed with proper CORS.	**Pass**
TC-008	Messaging	Real-time delivery of private message.	1. Open chat with User B. 2. Send "Hello".	Message appears natively for User A. User B receives message via WebSocket instantaneously.	Payload delivered over WebSocket in <100ms without duplication.	**Pass**
TC-009	Security	Prevent unauthorized access to Admin Dashboard.	1. Attempt to resolve `/admin` route as a non-admin user. 2. Monitor network calls.	UI denies access. API calls utilizing standard user JWT receive `403 Forbidden`.	Proper 403 returned, restricted panel not rendered.	**Pass**
TC-010	Activity	Test Follow Requests on Private Accounts	1. Create a "Private" account. 2. Have User B attempt to follow.	User B sees "Requested". Target receives follow request in notifications hub.	Privacy setting respected; request placed in queue properly.	**Pass**

---

8. Test Execution Results

All testing scenarios were executed during the pre-deployment quality assurance phases. The execution iterations were separated into parallel CI pipelines tracking individual modules.

- The Go backend test suites comprised **142 total unit and integration tests**, running securely inside internal mock staging deployments.
- The React frontend test suites comprised **86 component tests**.
- Playwright E2E suites successfully mapped **24 distinct critical paths**.

Systemic observation concluded that memory leaks generated during repeated 3D UI render cycles (holographic avatars) were resolved in the final patch version, resulting in stable test outcomes.

---

9. Defect Tracking

The platform follows a standardized rigid defect lifecycle matrix:

1. **Identification & Logging:** A defect is logged by a tester or flagged by CI execution via GitHub Issues with attachments including stack traces, screenshots, and reproduction steps.
2. **Triage & Assessment:** Development Leads assess the defect, assigning critical priority limits (e.g., Critical, High, Medium, Low) based on system impact.
3. **Assignment:** Assigned to the relevant engineer (`alexnikshith`, `Riteesh T M`, or `Kaushal-Loya`) for refactoring.
4. **Resolution (Fix):** Code is patched, tested locally, and submitted via a Pull Request.
5. **Re-Testing & Closure:** The CI pipeline runs regression tasks automatically on the targeted PR. If metrics pass, the defect is closed and merged to the main branch.

---

10. Risk Analysis

Several risks have been identified regarding the architectural stability of physical operations.

Risk Description	Probability	Impact	Mitigation Strategy
External AI Moderation Rate Limits	High	Medium	Implemented a constrained scan limit (max 10 recent posts) and dual-channel queuing architectures to offset Groq API over-utilization.
Federation Topology Fragmentation	Medium	High	Due to network partitioning or targeted blocking by external Mastodon nodes, payloads could drop. Handled via systematic ActivityPub retry-mechanisms upon 408/500 responses.
Database Over-Saturation (Ephemeral Data)	High	Low	Story analytics generate high-volume traffic. Ghost-data deletion mechanisms explicitly attached to cascade user deletions are utilized to wipe extraneous metric stores dynamically.

---

11. Test Summary Report

A final consolidation of the test results extracted from the latest build pipeline (`Branch: main`, `Environment: Staging`):

Scope	Executed	Passed	Failed	Skipped / Blocked	Pass Rate
Backend Integration	142	142	0	0	100%
Frontend Component	86	86	0	0	100%
Playwright E2E	24	24	0	0	100%
Federation Simulation	12	11	0	1 (Remote Node Offline)	91.6%
Total Aggregation	**264**	**263**	**0**	**1**	**99.6%**

System Reliability Outline: The software operates cohesively without introducing critical blocking states. P2P communication paradigms are highly responsive.

---

12. Conclusion

Based on the highly positive metrics aggregated within this test documentation phase (establishing a total system pass rate of **99.6%**), the "Federated Social Networking Platform with Decentralized Identity" exhibits exceptional architectural robustness. The security, moderation, content exchange, and federated protocols align strictly with user specifications.

Sign-off Decision: The system is evaluated and certified as **READY FOR DEPLOYMENT** to production environments.